Lab 12

Task:

Implement a Traffic Light controller using FSM.

```
module TrafficLightController(clk, rst, vehicle, timer, highwaySignal, farmSignal);
input clk, rst, vehicle;
input [3:0] timer;
output reg [2:0] highwaySignal, farmSignal;

reg [1:0] state, next;

parameter HG_FR = 2'b00; // Highway Green, Farm Red
parameter HY_FR = 2'b01; // Highway Yellow, Farm Red
parameter HR_FG = 2'b10; // Highway Red, Farm Green
parameter HR_FY = 2'b11; // Highway Red, Farm Yellow

always @(posedge clk or posedge rst) begin
if (rst)
state <= HG_FR;
else
state <= next;
end

always @(*) begin
case (state)
HG_FR: begin
highwaySignal = 3'b001;
farmSignal = 3'b100;
if (vehicle)
next = HY_FR;
else
next = HG_FR;
end

HY_FR: begin
highwaySignal = 3'b010;
farmSignal = 3'b100;
if (timer >= 3)
next = HR_FG;
else
next = HY_FR;
end

HR_FG: begin
```

```
highwaySignal = 3'b100;
farmSignal = 3'b001;
if (timer >= 10)
next = HR_FY;
else
next = HR_FG;
end

HR_FY: begin
highwaySignal = 3'b100;
farmSignal = 3'b010;
if (timer >= 3)
next = HG_FR;
else
next = HR_FY;
end

default: begin
highwaySignal = 3'b001;
farmSignal = 3'b100;
next = HG_FR;
end
endcase
end

endmodule
```

Task2:

```
module TimerModule(clk, rst, enable, second_count);
input clk, rst, enable;
output reg [3:0] second_count;
reg [25:0] count;
```

```
always @(posedge clk or posedge rst) begin
if (rst) begin
count <= 0;
second_count <= 0;
end else begin
if (enable) begin
if (count >= 49999999) begin
count <= 0;
second_count <= second_count + 1;
end else begin
count <= count + 1;
end
end else begin
count <= 0;
second_count <= 0;
end
end
end
endmodule

module TrafficLightController(clk, rst, vehicle, timer, highwaySignal, farmSignal);
input clk, rst, vehicle;
input [3:0] timer;
output reg [2:0] highwaySignal, farmSignal;
reg [1:0] state, next;

parameter HG_FR = 2'b00;
parameter HY_FR = 2'b01;
parameter HR_FG = 2'b10;
parameter HR_FY = 2'b11;

always @(posedge clk or posedge rst) begin
if (rst) state <= HG_FR;
else state <= next;
end

always @(*) begin
case (state)
HG_FR: begin
highwaySignal = 3'b001;
farmSignal = 3'b100;
if (vehicle) next = HY_FR; else next = HG_FR;
end
HY_FR: begin
```

```
highwaySignal = 3'b010;
farmSignal = 3'b100;
if (timer >= 3) next = HR_FG; else next = HY_FR;
end
HR_FG: begin
highwaySignal = 3'b100;
farmSignal = 3'b001;
if (timer >= 10) next = HR_FY; else next = HR_FG;
end
HR_FY: begin
highwaySignal = 3'b100;
farmSignal = 3'b010;
if (timer >= 3) next = HG_FR; else next = HR_FY;
end
default: begin
highwaySignal = 3'b001;
farmSignal = 3'b100;
next = HG_FR;
end
endcase
end
endmodule
```

TOP MODULE

```
module TrafficLightTop(clk, rst, vehicle, highwaySignal, farmSignal);
input clk, rst, vehicle;
output [2:0] highwaySignal, farmSignal;

wire [3:0] timer_out;
wire [1:0] current_state;
wire enable_timer;

TrafficLightController fsm(
.clk(clk),
.rst(rst),
.vehicle(vehicle),
.timer(timer_out),
.highwaySignal(highwaySignal),
.farmSignal(farmSignal)
);

assign enable_timer = (fsm.state != 2'b00); // FSM not in idle? enable timer
```

```
TimerModule timer(
.clk(clk),
.rst(rst),
.enable(enable_timer),
.second_count(timer_out)
);
endmodule
```